

Secure Comparison in (Constant) Two Rounds

Koji Nuida^{1,2} Satsuya Ohata^{1,3}

¹ Kyushu University, Fukuoka, 819-0395, Japan

² AIST, Tokyo, 135-0064, Japan

³ Byerlis Inc., Tokyo, 170-0013, Japan

June 21, 2026

Abstract

Secure comparison is a fundamental building block frequently employed in various applications of secure multiparty computation, such as secure machine learning. Such protocols based on secret sharing (SS) typically excel in throughput compared to garbled circuits (GC), but they historically suffer from higher (online) round complexity: while GC-based comparison ends in two rounds, the state-of-the-art SS-based (plaintext) comparison protocol requires three rounds (Lu et al., USENIX Security 2025). To break the barrier, in this paper we propose the first SS-based comparison protocol, built upon “round absorption” via multi-fan-in gates, to match the two-round complexity of GC with online bit complexity $O(n \log n)$ significantly lower than GC-based $O(\lambda n)$. We also propose the second two-round protocol, built upon a new optimization technique for multiplication, that addresses the drawback of $O(n^3)$ offline bit complexity in our first protocol and reduces it to $O(n^2)$ at the cost of increasing the online bit complexity to also $O(n^2)$.

keywords: Secure Multiparty Computation, Secure Comparison

1 Introduction

Secure multiparty computation (MPC) is a cryptographic framework that allows multiple mutually distrustful parties to jointly compute a function without revealing their private inputs. While MPC spans a broad spectrum of research directions, we focus on a fundamental primitive: secure comparison protocols, traditionally known as the Millionaires’ Problem [23]. Secure comparison protocols are frequently used in practical applications, and their high overhead is well-recognized. For example, a study on secure neural networks, CryptGPU [21], reported that operations involving comparison (specifically ReLU and Softmax) accounted for 42–73% of the total execution time (Table 5 in [21]). While numerous studies have focused on improving the efficiency of secure comparison protocols (cf. Table 1 below), it continues to be a dominant performance bottleneck in practical privacy-preserving applications. In particular, while protocols based on secret sharing (SS) are typically recognized for their superior efficiency in terms of both computation and communication volume (i.e., the total number of bits transferred), they suffer from higher round complexity than approaches based on other cryptographic primitives such as garbled circuits (GC). Specifically, although the state-of-the-art SS-based plaintext comparison protocol of Lu et al. [11] has achieved a constant three-round online complexity (as demonstrated in [11], such comparison over plaintexts can be extended to secret-shared inputs by sacrificing a single bit of the plaintext space), it is still higher than the GC-based two-round complexity. It has been a long-standing open problem to realize SS-based comparison achieving the same two-round complexity as GC.

Table 1: Comparison with the previous plaintext comparison protocols for n -bit integers; λ denotes the security parameter; μ denotes the bit length of a group element satisfying a suitable computational assumption (e.g., CDH assumption); $\log_2^*$ represents the iterated logarithm with base 2; $(\dagger)$ under correlated keys generation scheme which performs $O(2^n)$ times Hash locally; $(\dagger\dagger)$ under a trusted third-party dealer; $(\star)$ $\mathbf{x}_1 = 2n_2 + 8n_3 + 22n_4$ and $\mathbf{x}_2 = n_2 + n_3 + n_4$, where n_2 , n_3 , and n_4 denote the numbers of AND gates with 2, 3, and 4 inputs, respectively, in the bit extraction circuit; and most of the values (and remarks) for previous protocols are quoted from [11, Table 1].

Approach	Protocol	Offline	Online	
		Communication	Communication	# of rounds
GC-based	Yao [22, 24]	$2n\lambda$	$2n\lambda$	2
HE-based	GSV [8]	–	$18\mu n + 8\mu$	9
FSS-based	Half-Tree [10]	$(n + 2)\lambda (\dagger)$	$2n$	1
	DCF [2]	$4n(\lambda + 1) + \lambda + n (\dagger\dagger)$	$2n$	1
OT-based	Couteau [5]	$6\lambda n$	$8n + 2\log_2 n$	$4\log_2^* \lambda + 5$
	CrypTFlow2 [20]	–	$\lambda n + 14n$	$\log_2 n + 4$
	Lu et al. [11]	$\approx 2n\lambda \log_2(2n) + n \log_2^2 n$	$(2n + 3)(\log_2 n + 2)$	3
Generic 2PC	ABY [6]	$6\lambda n + 17\lambda + n$	$2\lambda n + 20n$	$\log_2 n + 5$
	ABY2.0 [19, 18]	$\mathbf{x}_1(\lambda + 1) + n (\star)$	$2\mathbf{x}_2 + n (\star)$	$\log_4 n + 1$
	Ours $\Pi_{\text{cmp},1}$ (§ 4)	$\approx 4n^3 + 2\mu + \lambda$ $+ (\mu + \lambda)(\log_2 n + \log_2^2 n)$	$2(n - 1)\log_2 n$ $+ 6n - 4$	2
	Ours $\Pi_{\text{cmp},2}$ (§ 5)	$\approx 3n^2 + 2\mu + \lambda$ $+ 2(n + \mu + \lambda)\log_2 n$	$2n^2 - 2$	2

1.1 Our Contribution

We propose the first SS-based comparison protocol with a constant two-round online complexity, solving the aforementioned open problem of achieving the same round complexity as GC-based protocols. Our protocol also maintains the $O(n \log n)$ online bit complexity of Lu et al.’s three-round protocol [11], which is significantly lower than the GC-based $O(\lambda n)$. A drawback of our protocol is the $O(n^3)$ offline bit complexity (in comparison to e.g., GC-based $O(\lambda n)$): to address the issue, we propose the second two-round protocol with offline bit complexity $O(n^2)$, while it increases the online bit complexity to also $O(n^2)$.

A summary of these protocols alongside existing works is provided in Table 1. We also compare the concrete efficiency of our proposed protocols with the state-of-the-art protocol by Lu et al. [11], where the parameters are set to $\lambda = 128$ and $\mu = 512$ (see Table 1 for the meaning of these parameters). See Table 2 for the result. The table shows that our first proposed protocol $\Pi_{\text{cmp},1}$ has the smaller online round complexity than Lu et al.’s protocol while the online communication volume is slightly increased. Our second proposed protocol $\Pi_{\text{cmp},2}$ also has the smaller online round complexity and excels in the total (offline + online) communication volume.

1.2 Technical Overview

Unlike approaches based on GC or FSS—which often face limitations in terms of bit complexity or setup costs—designing constant-round SS-based comparison protocols was a non-trivial challenge: traditional strategies, such as the divide-and-conquer approach or those utilizing parallel prefix adders to determine the sign of the difference between two integers, typically require $O(\log n)$ rounds. Against this backdrop, Lu et al. [11] introduced a new design paradigm and achieved a constant three-round complexity. However, we are aiming at achieving the same two-round complexity as GC, and this seems difficult when following Lu

Table 2: Efficiency comparison between our protocols and the existing state-of-the-art protocol. The parameters (see Table 1) are set to $n = 64$, $\lambda = 128$, and $\mu = 512$.

Protocol	Offline	Online	
	Communication	Communication	# of rounds
Lu et al. [11]	14.3 KiB	131 B	3
Ours $\Pi_{\text{cmp},1}$ (§ 4)	131.4 KiB	142 B	2
Ours $\Pi_{\text{cmp},2}$ (§ 5)	2.7 KiB	1024 B	2

et al.’s strategy as explained below, necessitating a new technique to break the barrier.

Specifically, Lu et al.’s approach is based on a classic observation: the comparison of two integers can be reduced to identifying the leftmost differing bit position. For example, suppose that P_0 holds an integer $x = 10101 \cdots$ and P_1 holds $y = 10110 \cdots$ (of the same bit lengths). Then the fact that they first differ at the fourth bit determines the result $x < y$. The high-level strategy of the protocol by Lu et al. [11] can be summarized as follows:

1. Convert each bit into arithmetic shares using the bit2A protocol (Round 1).
2. Execute the oblivious selective zero check (OZC) protocol (Round 2-3):
 - Use prefix sums and algebraic manipulations to output a vector to P_1 such that only the leftmost differing bit position is zero, while all other positions are non-zero.
 - Identify the value of x (held by P_0) at the position corresponding to this zero entry.

Here we observe that if the bit2A at the first round could be “absorbed” into the subsequent OZC protocol (or if the round complexity of OZC itself could be reduced), then we would achieve a desired two-round complexity. However, as the OZC protocol is not flexible, such a “round absorption” (or round reduction) seems hopeless. Accordingly, we need to depart from the OZC-based approach.

Protocol 1 (Section 4): Achieving Two-Round Complexity via “Round Absorption”. We first give an auxiliary generic construction of a three-round comparison protocol for n -bit inputs $x = (x_{n-1} \cdots x_1 x_0)_2$ and $y = (y_{n-1} \cdots y_1 y_0)_2$ using any one-round equality testing protocol for $O(\log n)$ -bit inputs (having the same three-round complexity as Lu et al.’s protocol [11] but with a different structure). An outline of our construction is as follows. The starting point is the following natural idea: we have $x < y$ if and only if there is some (in fact, a unique) index i with the property that $(x_{n-1} \cdots x_{i+1})_2 = (y_{n-1} \cdots y_{i+1})_2$ and $x_i < y_i$. Now the check for the former condition $(x_{n-1} \cdots x_{i+1})_2 = (y_{n-1} \cdots y_{i+1})_2$ is reduced (in Round 1) to the equality testing of two $O(\log n)$ -bit integers via the dimension reduction technique of Lu et al. [11, Section 3.2]. The latter equality testing is solved via an auxiliary one-round equality testing protocol in Round 2. Then in Round 3, we combine the result of the aforementioned equality testing with the result of $x_i < y_i$ (which is computed in a previous round in parallel) to obtain the final result.

Specifically, our generic construction proceeds as follows, where $\mathbf{1}\{\text{Cond}\} := 1$ if Cond is true and $\mathbf{1}\{\text{Cond}\} := 0$ if Cond is false:

(Round 1) For each, say the i -th input bit, compute an arithmetic share of $\text{neq}_{[n-1,i+1]} := \sum_{j=i+1}^{n-1} \mathbf{1}\{x_j \neq y_j\}$ and a Boolean share of $\text{lt}_i := \mathbf{1}\{x_i < y_i\}$.

(Round 2) For each i , compute a Boolean share of $Z_{i+1} := \mathbf{1}\{(x_{n-1}, \dots, x_{i+1}) = (y_{n-1}, \dots, y_{i+1})\} = \mathbf{1}\{\text{neq}_{[n-1,i+1]} = 0\}$ by using any one-round equality testing protocol.

(Round 3) Compute a Boolean share of $\mathbf{1}\{x < y\} = \text{lt}_{n-1} \oplus \bigoplus_{i=0}^{n-2} (Z_{i+1} \wedge \text{lt}_i)$.

Although the round complexity of this protocol is the same as Lu et al.’s protocol, it has an advantage that our construction is more flexible because any one-round equality testing protocol can be used inside the protocol.

Then we instantiate the internal one-round equality testing protocol as in Section 3.1. In this instantiation, a share of each Z_{i+1} is computed in Round 2 by an n_0 -fan-in AND with $n_0 = \lceil \log_2 n \rceil$. Consequently, the AND operation $Z_{i+1} \wedge \text{lt}_i$ in Round 3 can be absorbed into Round 2 using an $(n_0 + 1)$ -fan-in AND, resulting in a two-round protocol. We emphasize that such a “round absorption” technique is not applicable to Lu et al.’s comparison protocol due to its different structure. Moreover, the existing one-round equality testing protocol of Lu et al. [11, Section 3] does not enable the round absorption, therefore we had to develop a new one-round equality testing protocol for our purpose.

Protocol 2 (Section 5): Balancing Offline/Online Bit Complexity via New Multiplication Technique for Trivial Shares. In order to address the issue of the higher offline bit complexity in Protocol 1 due to the use of larger fan-in ANDs, here we instantiate the internal equality testing protocol in our generic construction as in Section 3.2 (instead of Section 3.1). Now each bit Z_{i+1} is computed as XOR of plain ANDs, say $Z_{i+1} = \bigoplus_j X_{i+1,j} \wedge Y_{i+1,j}$ (see Section 3.2 for a precise definition of these terms). Consequently, a naive application of the same technique as in Protocol 1 already achieves two-round complexity using 3-fan-in ANDs $X_{i+1,j} \wedge Y_{i+1,j} \wedge \text{lt}_i$ instead of $(n_0 + 1)$ -fan-in ANDs.

For more efficiency improvement, here we observe that the bits $X_{i+1,j}$ and $Y_{i+1,j}$ are known as plaintexts by party P_0 and party P_1 , respectively. Moreover, we have $\text{lt}_i = (\neg x_i) \wedge y_i$, and the bits $\neg x_i$ and y_i are also known as plaintexts by party P_0 and party P_1 , respectively. Then, by arranging the computation as

$$X_{i+1,j} \wedge Y_{i+1,j} \wedge \text{lt}_i = (X_{i+1,j} \wedge (\neg x_i)) \wedge (Y_{i+1,j} \wedge y_i) ,$$

each of the values in two large parentheses in the right-hand side can be locally computed by a single party, therefore the overall computation can be done by plain ANDs (instead of 3-fan-in ANDs) for “trivially” shared bits. This removal of larger fan-in ANDs reduces the offline bit complexity in contrast to Protocol 1, at the cost of increasing the online bit complexity due to the larger number of AND operations.

1.3 Related Work

Generally, garbled circuits (GC) [22, 24] provide the baseline for constant (two)-round comparison but incur high bandwidth costs $O(\lambda n)$. Constant-round comparison protocols based on (fully) homomorphic encryption (HE) are also proposed [8, 16, 4], but they either have higher round complexity or suffer from large computational and communication overhead. While various SS-based approaches have attempted to reduce rounds, they often rely on relaxed models (e.g., the five-round protocol in [13] based on the client-aided model [12]) or expensive pre-computation (e.g., the use of multi-fan-in gates [17, 19, 7] or function secret sharing (FSS) [10, 2]). Recently, Lu et al. [11] established the state-of-the-art for SS-based comparison, achieving three rounds. However, matching the two-round efficiency of GC within the SS framework has remained an elusive goal. Our Protocols 1 and 2 directly address this gap.

1.4 Paper Organization

Section 2 describes the basic notation and the existing techniques used in this paper. Section 3 introduces the equality testing protocols that serve as the foundation for our comparison protocols. In Sections 4 and 5, we present our comparison protocols along with their bit complexity analysis. Section 6 demonstrates the practical utility of our proposed comparison protocols through a case study on secure neural network inference. Finally, Section 7 concludes the paper and outlines future research directions.

Table 3: Functionalities and costs for share conversion.

Protocol	Input		Output	Online communication	
	Main	Auxiliary		Rounds	Bits
bit2T	$[b]^B$	$[\beta]^B$ for some $\beta \in \{0, 1\}$	$\langle b \rangle$ consistent with $[\beta]^B$	1	2
T2A	$\langle b \rangle$ $= ([\delta_b]_0^B, [\delta_b]_1^B; \Delta_b)$	$[\delta_b]^A$	$[b]^A$	0	0
bit2A $= \text{T2A} \circ \text{bit2T}$	$[b]^B$	$[\beta]^B, [\beta]^A$ for some $\beta \in \{0, 1\}$	$[b]^A$	1	2

2 Preliminaries

In this paper, let λ denote the security parameter. Let $\parallel$ denote the concatenation of strings. Let $\lceil r \rceil$ for $r \in \mathbb{R}$ denote the minimum integer k satisfying $k \geq r$. For any condition **Cond**, let $\mathbf{1}\{\text{Cond}\} := 1$ if **Cond** is true and $\mathbf{1}\{\text{Cond}\} := 0$ if **Cond** is false. For integers $n \geq 0$ and $k > 0$, let $n \bmod k$ denote the remainder of n modulo k taken from the interval $0 \leq (n \bmod k) \leq k - 1$. We write $\mathbb{Z}_N := \mathbb{Z}/N\mathbb{Z}$ in short, and write $[k] := \{0, 1, \dots, k - 1\}$ for an integer $k \geq 1$. Let $(a_{\ell-1} \dots a_1 a_0)_2$ denote the binary expression of an integer $\sum_{j=0}^{\ell-1} a_j 2^j$.

2.1 Model for MPC

We deal with secure two-party computation (2PC) protocols between parties P_0 and P_1 in the *preprocessing model*, consisting of an input-independent *offline phase* and an input-dependent *online phase*. The protocols in this paper are purely 2PC protocols, unlike the client-aided model [12] where the offline phase is supposed to be executed by a trusted client other than the two parties. Our main aim is to reduce the communication complexity (the numbers of communication rounds and communication bits) for the online phase, while we also require the offline phase to still have polynomial computational/communication complexity (unlike FSS-based approaches [10] using exponential offline computation).

We assume that the two parties are *semi-honest*, and follow the standard security notion in [9]. In this setting, owing to Composition Theorem [9, Theorem 7.3.3], for any protocol Π involving some subprotocol Π' , if Π is secure assuming oracle access to the ideal functionality $\mathcal{F}'$ of Π' ($\mathcal{F}'$ -*hybrid model*), then Π is secure provided Π' is securely implemented. Most of the protocols proposed in this paper can be easily proven secure in such a hybrid model.

2.2 Secret Sharing

We use the following three kinds of 2-out-of-2 secret sharing (SS) schemes. Some of these shares can be converted into other kinds of shares by using protocols as in [19]; we describe these protocols in Section 2.3 below for the sake of completeness. The functionality and complexity of these share conversion protocols are summarized in Table 3 (where the ‘T’ stands for “triplets”).

2.2.1 $\square^B$ -Sharing

It is the ordinary additive SS scheme for bits: for a share ($\square^B$ -share) $[b]^B := ([b]_0^B, [b]_1^B)$ for $b \in \{0, 1\}$, we have $b = [b]_0^B \oplus [b]_1^B$. we assume that $[b]_0^B$ is held by party P_0 only and $[b]_1^B$ is held by party P_1 only. Now we introduce the following terminology:

Definition 1. We call a share for $b \in \{0, 1\}$ of the form $([b]_0^B, [b]_1^B) = (b, 0)$ (resp. $([b]_0^B, [b]_1^B) = (0, b)$) the P_0 -side (resp. P_1 -side) *trivial* $\square^B$ -share for b .

2.2.2 $\square^A$ -Sharing

It is the ordinary additive SS scheme for integers in $\mathbb{Z}_N$ for a suitably chosen integer N : for a share ($\square^A$ -share) $[x]^A := ([x]_0^A, [x]_1^A)$ for $x \in \mathbb{Z}_N$, we have $x = [x]_0^A + [x]_1^A \bmod N$. We assume that $[x]_0^A$ is held by party P_0 only and $[x]_1^A$ is held by party P_1 only.

2.2.3 $\langle \rangle$ -Sharing ([19])

For $b \in \{0, 1\}$, we let $[\delta_b]_0^B$, $[\delta_b]_1^B$, and Δ_b be a triplet of bits satisfying the relation $[\delta_b]_0^B \oplus [\delta_b]_1^B \oplus \Delta_b = b$, and let $[\delta_b]_0^B$ be held by party P_0 only, $[\delta_b]_1^B$ be held by party P_1 only, and Δ_b be held by both parties. We write the share for b as $\langle b \rangle = ([\delta_b]_0^B, [\delta_b]_1^B; \Delta_b)$, and call it a $\langle \rangle$ -share for b . We note that the pair $([\delta_b]_0^B, [\delta_b]_1^B)$ can be viewed as a $\square^B$ -share for the bit $\delta_b := [\delta_b]_0^B \oplus [\delta_b]_1^B$. Now we introduce the following terminology:

Definition 2. For a $\langle \rangle$ -share $\langle b \rangle$ and a $\square^B$ -share $[\beta]^B$, we say that $\langle b \rangle$ is *consistent with* $[\beta]^B$ if $[\delta_b]_0^B = [\beta]_0^B$ and $[\delta_b]_1^B = [\beta]_1^B$.

2.3 Share Conversion Protocols

Here we describe the protocols for converting three types of shares given in [19].

bit2T Given a $\square^B$ -share $[b]^B$ and an auxiliary $\square^B$ -share $[\beta]^B$, we can consider the following protocol of converting $[b]^B$ to a $\langle \rangle$ -share $\langle b \rangle$ that is consistent with $[\beta]^B$ (in the sense of Definition 2):

1. For $i = 0, 1$, P_i sends $\alpha_i := [b]_i^B \oplus [\beta]_i^B$ to P_{1-i} .
2. For $i = 0, 1$, P_i locally sets $[\delta_b]_i^B \leftarrow [\beta]_i^B$ and $\Delta_b \leftarrow \alpha_0 \oplus \alpha_1$.

We write the protocol as **bit2T** $([b]^B; [\beta]^B)$, or simply **bit2T** $([b]^B)$ in which case it is implicitly assumed that some fresh $\square^B$ -share $[\beta]^B$ is used as an auxiliary input.

T2A Given a $\langle \rangle$ -share $\langle b \rangle = ([\delta_b]_0^B, [\delta_b]_1^B; \Delta_b)$ for a bit b and an auxiliary $\square^A$ -share $[\delta_b]^A$ for the value δ_b viewed as an integer, we can consider the following protocol of converting $\langle b \rangle$ to a $\square^A$ -share $[b]^A$ for the same value b viewed as an integer:

1. P_0 locally sets $[b]_0^A \leftarrow [\delta_b]_0^A - 2\Delta_b \cdot [\delta_b]_0^A$, and P_1 locally sets $[b]_1^A \leftarrow [\delta_b]_1^A + \Delta_b - 2\Delta_b \cdot [\delta_b]_1^A$.

We write the protocol as **T2A** $(\langle b \rangle; [\delta_b]^A)$, or simply **T2A** $(\langle b \rangle)$ in which case it is implicitly assumed that some fresh $\square^A$ -share $[\delta_b]^A$ is used as an auxiliary input.

bit2A Given a $\square^B$ -share $[b]^B$ for a bit b and auxiliary $\square^B$ -share $[\beta]^B$ and $\square^A$ -share $[\beta]^A$ for some common value $\beta \in \{0, 1\}$, we can convert $[b]^B$ to a $\square^A$ -share $[b]^A$ for the same value b viewed as an integer, by composing the protocols **bit2T** and **T2A**. We write the protocol as **bit2A** $([b]^B; [\beta]^B, [\beta]^A)$, or simply **bit2A** $([b]^B)$. Precisely, the protocol is given as follows:

1. The parties compute $\langle b \rangle \leftarrow \text{bit2T}([b]^B; [\beta]^B)$.
2. The parties locally compute $[b]^A \leftarrow \text{T2A}(\langle b \rangle; [\beta]^A)$.

We note that the auxiliary inputs $[\beta]^B$ and $[\beta]^A$, where the plaintext space for $[\beta]^A$ is $\mathbb{Z}_N$, can be generated in the offline phase by using $\binom{2}{1}$ -OT $_{\lceil \log_2 N \rceil}$ (see Section 2.4 for the notation) as follows:

1. P_0 chooses random elements $[\beta]_0^B \in \{0, 1\}$ and $[\beta]_0^A \in \mathbb{Z}_N$. P_1 chooses a random bit $[\beta]_1^B$.
2. P_0 as the sender and P_1 as the receiver execute $\binom{2}{1}$ -OT $_{\lceil \log_2 N \rceil}$, where P_0 's input is $(([\beta]_0^B - [\beta]_0^A) \bmod N, (1 - [\beta]_0^B - [\beta]_0^A) \bmod N)$ and P_1 's input is $[\beta]_1^B$. Then, given P_1 's output z of the OT, P_1 sets $[\beta]_1^A \leftarrow z$.

Table 4: Functionalities and costs of AND operations.

Operation	Input	Output	Offline cost	Online communication	
				Rounds	Bits
plain AND	$[X]^B, [Y]^B$	$[X \wedge Y]^B$	$\binom{4}{1}$ -OT ₁	1	4
half-AND	$[X]^B = (X, 0),$ $[Y]^B = (0, Y)$	$[X \wedge Y]^B$	$\binom{2}{1}$ -OT ₁	1	2

2.4 Oblivious Transfer

A 1-out-of- k *oblivious transfer* (OT) between a sender and a receiver is a 2PC protocol where the sender has input $x_0, \dots, x_{k-1}$, the receiver has input $r \in \{0, \dots, k-1\}$, and after the protocol, the sender obtains no information on the receiver's input and the receiver obtains x_r and no other information on the sender's input. In this paper, n parallel executions of 1-out-of- k OTs for ℓ -bit inputs for the sender is denoted by $\binom{k}{1}$ -OT _{ℓ} ^{n} . We sometimes omit the superscript ' n ' when $n = 1$. Let $\mathcal{F}_{\text{OT}}$ denote the ideal functionality for OT with suitable parameters.

Here we suppose that an OT is executed by a standard technique as in [14] based on a vector oblivious linear evaluation (VOLE) correlation, which is generated according to [3] where the internal small OTs are executed with the method of [15]. Now the number of communication bits for executing a $\binom{k}{1}$ -OT _{ℓ} (given a VOLE correlation) is $2^{\lceil \log_2 k \rceil} \ell + \lceil \log_2 k \rceil$ bits, while it spends $\lceil \log_2 k \rceil$ dimensions for a VOLE correlation. On the other hand, to generate a VOLE correlation of dimension d , the number of communication bits is approximately $2\mu + \lambda + (\mu + \lambda) \log_2 d$ bits where $\mu = \mu_\lambda$ denotes the bit length of an element of a group for which the CDH assumption holds. See Appendix A for details.

2.5 AND Operations for $\llbracket^B$ -Shares

Given $\llbracket^B$ -shares $[X]^B$ and $[Y]^B$ as inputs, a plain AND operation to compute a $\llbracket^B$ -share $[X \wedge Y]^B$ can be realized by using the following plain Beaver Triple (BT) [1]: $[a]^B, [b]^B$, and $[c]^B$, where $c = a \wedge b$. A plain BT $([a]^B, [b]^B, [c]^B)$ can be generated via $\binom{4}{1}$ -OT₁ in the following manner, which is secure in the $\mathcal{F}_{\text{OT}}$ -hybrid model:

1. P_0 , as the sender, chooses random bits $[a]_0^B, [b]_0^B$, and $[c]_0^B$, and sets the $([a]_1^B \parallel [b]_1^B)$ -th input for $\binom{4}{1}$ -OT₁ to be $([a]_0^B \oplus [a]_1^B) \wedge ([b]_0^B \oplus [b]_1^B) \oplus [c]_0^B$ for each $[a]_1^B, [b]_1^B \in \{0, 1\}$.
2. P_1 , as the receiver, chooses random bits $[a]_1^B$ and $[b]_1^B$, and sets the input for $\binom{4}{1}$ -OT₁ to be $[a]_1^B \parallel [b]_1^B$. Then P_1 writes the output of the OT as $[c]_1^B$.

On the other hand, when $[X]^B$ is a P_0 -side trivial $\llbracket^B$ -share and $[Y]^B$ is a P_1 -side trivial $\llbracket^B$ -share (see Definition 1 for the terminology), the AND operation can be realized more efficiently by using "arithmetic blinding" [17], with a modified BT of the form $[a]^B = (a, 0)$, $[b]^B = (0, b)$, and $[c]^B$, where $c = a \wedge b$. We call this AND operation a *half-AND*. Similarly to the case of a plain BT described above, a modified BT $([a]^B, [b]^B, [c]^B)$ for half-AND, where $[a]^B = (a, 0)$, $[b]^B = (0, b)$, and $c = a \wedge b$, can be generated via $\binom{2}{1}$ -OT₁ where the sender's b -th input is a value corresponding to $[c]_1^B$.

The functionalities and the costs of those (plain or modified) AND operations are given in Table 4. We may write a plain AND operation and a half-AND operation simply by using the binary operation symbol $\wedge$.

2.6 Multi-fan-in AND for $\langle \rangle$ -Shares

We use an N -fan-in AND operation for $\langle \rangle$ -shares $\langle X_0 \rangle, \langle X_1 \rangle, \dots, \langle X_{N-1} \rangle$ described in [19, Section 3.1.4], which outputs a $\llbracket^B$ -share of the bit $X_0 \wedge X_1 \wedge \dots \wedge X_{N-1}$. Its auxiliary input is a so-called Beaver triple extension [17], N-BTE in short, which is a collection of $\llbracket^B$ -shares of the form $([a_S]^B)_{\emptyset \neq S \subseteq [N]}$ with the

relations $a_S = \bigwedge_{i \in S} a_{\{i\}}$ for each non-empty subset $S \subseteq [N]$. Moreover, it should satisfy that for each $i = 0, 1, \dots, N-1$, $\langle X_i \rangle$ is consistent with $[a_{\{i\}}]^B$ (see Definition 2 for the terminology).

The protocol proceeds as follows: by letting P_0 compute

$$[Z]_0^B \leftarrow \bigoplus_{\emptyset \neq S \subseteq [N]} \left([a_S]_0^B \wedge \bigwedge_{i \in [N] \setminus S} \Delta_{X_i} \right),$$

and letting P_1 compute

$$[Z]_1^B \leftarrow \left(\bigwedge_{i \in [N]} \Delta_{X_i} \right) \oplus \bigoplus_{\emptyset \neq S \subseteq [N]} \left([a_S]_1^B \wedge \bigwedge_{i \in [N] \setminus S} \Delta_{X_i} \right),$$

it can be verified by a straightforward calculation that $[Z]^B := ([Z]_0^B, [Z]_1^B)$ is a correct $\llbracket^B$ -share of the bit $X_0 \wedge X_1 \wedge \dots \wedge X_{N-1}$. Given $\langle \rangle$ -shares satisfying the condition above as the main input, the protocol above itself requires no online communication. We note that if the main input is a collection of $\llbracket^B$ -shares, then one-round online communication is required to convert the $\llbracket^B$ -shares to $\langle \rangle$ -shares consistent with a given BTE.

As for the offline cost, in a way similar to generating (plain or modified) BTs as in Section 2.5, an N-BTE $([a_S]^B)_{\emptyset \neq S \subseteq [N]}$ can be generated via $\binom{2^N}{1}$ -OT $_{2^N-N-1}$, where the input of P_1 as a receiver is $[a_{\{0\}}]_1^B \parallel [a_{\{1\}}]_1^B \parallel \dots \parallel [a_{\{N-1\}}]_1^B$ and the $([a_{\{0\}}]_1^B \parallel [a_{\{1\}}]_1^B \parallel \dots \parallel [a_{\{N-1\}}]_1^B)$ -th input of P_0 as a sender is a collection of the bits corresponding to $[a_S]_1^B$ over the $2^N - N - 1$ subsets $S \subseteq [N]$ with $|S| \geq 2$.

3 Auxiliary One-Round Equality Testing Protocols

In this section, we give two auxiliary equality testing protocols for plaintexts with one online communication round, which will be used in our first and second comparison protocols, respectively. Although the protocols here for n_0 -bit inputs require either an n_0 -fan-in AND or $\Theta(2^{n_0})$ -bit communication, it will not affect the polynomial complexity of the main protocols with n -bit inputs because these equality testing protocols are used with parameter $n_0 = O(\log n)$ owing to the fact that the bit length of the sum of all bits in an n -bit input is $O(\log n)$.

3.1 Our First Protocol

Given P_0 's input $x = (x_{n_0-1} \dots x_1 x_0)_2$ and P_1 's input $y = (y_{n_0-1} \dots y_1 y_0)_2$, our first protocol implements a natural idea “ $x = y$ if and only if $x_i = y_i$ for all positions i ” via multi-fan-in AND. Precisely, given also an n_0 -BTE $([a_S]^B)_{\emptyset \neq S \subseteq [n_0]}$ as an auxiliary input, the first protocol proceeds as follows:

1. For $i = 0, 1, \dots, n_0 - 1$, P_0 sets $[\mathbf{eq}_i]_0^B \leftarrow x_i \oplus 1$; P_1 sets $[\mathbf{eq}_i]_1^B \leftarrow y_i$; and the parties compute $\langle \mathbf{eq}_i \rangle \leftarrow \text{bit2T}([\mathbf{eq}_i]_0^B; [\mathbf{eq}_i]_1^B)$.
2. The parties locally compute an n_0 -fan-in AND $[\mathbf{eq}]^B \leftarrow \bigwedge_{i=0}^{n_0-1} \langle \mathbf{eq}_i \rangle$ by using the BTE $([a_S]^B)_{\emptyset \neq S \subseteq [n_0]}$, and output $[\mathbf{eq}]^B$.

The correctness follows from the fact that we have $\mathbf{eq}_i = \mathbf{1}\{x_i = y_i\}$ for every i and we have $x = y$ if and only if $x_i = y_i$ for every i . The protocol requires 1 round for the first step (for bit2T) and no communication for the second step (cf. Section 2.6), therefore 1 online round in total. The number of online communication bits is $2n_0$ bits (for n_0 executions of bit2T), and the number of offline communication bits is $2^{n_0}(2^{n_0} - n_0 - 1) + n_0 = O((2^{n_0})^2)$ bits (except for the setup of a VOLE correlation) for executing $\binom{2^{n_0}}{1}$ -OT $_{2^{n_0}-n_0-1}$ in generating the BTE.

3.2 Our Second Protocol

Let the parties' inputs be $x \in \mathbb{Z}_N$ and $y \in \mathbb{Z}_N$ for some parameter N . Our second protocol is based on the following idea: let $E_0 = (E_{0,0}, E_{0,1}, \dots, E_{0,N-1})$ be a one-hot vector encoding of x , i.e., $E_{0,i} = 1$ if $i = x$ and $E_{0,i} = 0$ if $i \neq x$. Similarly, let $E_1 = (E_{1,0}, E_{1,1}, \dots, E_{1,N-1})$ be a one-hot vector encoding of y . Then the inner product of E_0 and E_1 gives the bit $\mathbf{1}\{x = y\}$. Precisely, the second protocol proceeds as follows:

1. For $i = 0, 1, \dots, N-1$, P_0 sets $[E_{0,i}]_0^B \leftarrow \mathbf{1}\{x = i\}$ and $[E_{1,i}]_0^B \leftarrow 0$; and P_1 sets $[E_{0,i}]_1^B \leftarrow 0$ and $[E_{1,i}]_1^B \leftarrow \mathbf{1}\{y = i\}$.
2. The parties compute $[\text{eq}]^B \leftarrow \bigoplus_{i=0}^{N-1} [E_{0,i}]^B \wedge [E_{1,i}]^B$ by half-ANDs (see Section 2.5), and output $[\text{eq}]^B$.

The protocol requires 1 online round (for half-ANDs in the second step) in total. For the correctness, we first note that for each $i = 0, 1, \dots, N-1$, we have $E_{0,i} \wedge E_{1,i} = 1$ if and only if $x = i = y$. Therefore, if $x \neq y$ then we have $E_{0,i} \wedge E_{1,i} = 0$ for every i , while if $x = y$ then we have $E_{0,i} \wedge E_{1,i} = 1$ for precisely one i , i.e., $i = x (= y)$. Hence we have $\text{eq} = \mathbf{1}\{x = y\}$, as desired. The number of online communication bits is $2N = O(2^{n_0})$ bits for computing N half-ANDs in the second step, and the number of offline communication bits is $3N = O(2^{n_0})$ bits (except for the setup of a VOLE correlation) for executing $(\binom{2}{1})\text{-OT}_1^N$ in generating N BTs for half-ANDs.

4 Our First Proposed Comparison Protocol $\Pi_{\text{cmp},1}$

4.1 A Three-Round Generic Construction

First, we give a three-round comparison protocol using any one-round equality testing protocol as a subprotocol. Let P_0 's input be $x = (x_{n-1} \dots x_1 x_0)_2$ and P_1 's input be $y = (y_{n-1} \dots y_1 y_0)_2$. Our protocol here is based on the following property: we have $x < y$ if and only if there is some (in fact, a unique) index i for which $x_j = y_j$ for every $i+1 \leq j \leq n-1$ and $x_i < y_i$. Now the former property, i.e., $x_j = y_j$ for every $i+1 \leq j \leq n-1$, is checked by compressing at most $n-1$ bits $\mathbf{1}\{x_j \neq y_j\}$ into a single $O(\log n)$ -bit integer and using a one-round equality testing protocol.

Precisely, the protocol is described as follows:

1. (a) For $i = n-1, \dots, 2, 1$, P_0 sets $[\text{neq}_i]_0^B \leftarrow x_i$; P_1 sets $[\text{neq}_i]_1^B \leftarrow y_i$; and the parties compute $[\text{neq}_i]^A \leftarrow \text{bit2A}([\text{neq}_i]^B)$ (Round 1) where the plaintext space for $[\text{neq}_i]^A$ is set to $\mathbb{Z}_n$.
 (b) For $i = n-1, \dots, 1, 0$, P_0 sets $[\neg x_i]_0^B \leftarrow x_i \oplus 1$ and $[y_i]_0^B \leftarrow 0$; P_1 sets $[\neg x_i]_1^B \leftarrow 0$ and $[y_i]_1^B \leftarrow y_i$; and the parties compute $[\text{lt}_i]^B \leftarrow [\neg x_i]^B \wedge [y_i]^B$ (Round 1).
2. (a) For $i = n-1, \dots, 2, 1$, the parties locally compute $[\text{neq}_{[n-1,i]}]^A \leftarrow \sum_{j=i}^{n-1} [\text{neq}_j]^A$.
 (b) Then for $i = n-1, \dots, 2, 1$, the parties compute a $\mathbb{Z}_n$ -share $[Z_i]^B$ of the bit $Z_i := \mathbf{1}\{[\text{neq}_{[n-1,i]}]_0^A = (n - [\text{neq}_{[n-1,i]}]_1^A) \bmod n\}$ by using some one-round equality testing protocol (Round 2).
3. The parties compute $[\text{lt}]^B \leftarrow [\text{lt}_{n-1}]^B \oplus \bigoplus_{i=0}^{n-2} ([Z_{i+1}]^B \wedge [\text{lt}_i]^B)$ (Round 3) and output $[\text{lt}]^B$.

Correctness. First note that $\text{neq}_i = \mathbf{1}\{x_i \neq y_i\}$ and $\text{lt}_i = \mathbf{1}\{x_i < y_i\}$ for every i . Then for each $i = n-1, \dots, 2, 1$, we have $Z_i = 1$ (or equivalently, $\text{neq}_{[n-1,i]} = 0$ in $\mathbb{Z}_n$) if and only if $\text{neq}_j = 0$ (i.e., $x_j = y_j$) for every $j = i, i+1, \dots, n-1$, namely $(x_{n-1}, \dots, x_{i+1}, x_i) = (y_{n-1}, \dots, y_{i+1}, y_i)$. Now the final step simply implements the comparison under the lexicographic ordering. Hence we have $\text{lt} = \mathbf{1}\{x < y\}$.

4.2 Our Proposed Two-Round Protocol

When instantiating the protocol above with the equality testing protocol in Section 3.1, the second round becomes an n_0 -fan-in AND with $n_0 := \lceil \log_2 n \rceil$ to compute each $[Z_i]^B$. Now the AND operation $[Z_{i+1}]^B \wedge [\text{lt}_i]^B$ in the third round can be absorbed into the former n_0 -fan-in AND, reducing one communication round. This results in a two-round comparison protocol $\Pi_{\text{cmp},1}$ as in Protocol 1.

Protocol 1 Our First Two-Round Plaintext Comparison Protocol $\Pi_{\text{cmp},1}$

Input: P_0 has input $x = (x_{n-1} \cdots x_1 x_0)_2$, and P_1 has input $y = (y_{n-1} \cdots y_1 y_0)_2$

(Parameter: $n_0 := \lceil \log_2 n \rceil$)

Output: $\llbracket^B$ -share $\llbracket \mathbf{1}\{x < y\} \rrbracket^B$ of the bit $\mathbf{1}\{x < y\}$

```

1: for  $i = n - 1, \dots, 2, 1$  do
2:   Locally set  $\llbracket \text{neq}_i \rrbracket_0^B \leftarrow x_i$  and  $\llbracket \text{neq}_i \rrbracket_1^B \leftarrow y_i$ 
3:   Compute  $\llbracket \text{neq}_i \rrbracket^A \leftarrow \text{bit2A}(\llbracket \text{neq}_i \rrbracket^B)$  where the plaintext space for  $\llbracket \text{neq}_i \rrbracket^A$  is set to  $\mathbb{Z}_n$ 
4: end for
5: for  $i = n - 1, \dots, 1, 0$  do
6:   Locally set  $\llbracket \neg x_i \rrbracket_0^B \leftarrow x_i \oplus 1$ ,  $\llbracket \neg x_i \rrbracket_1^B \leftarrow 0$ ,  $\llbracket y_i \rrbracket_0^B \leftarrow 0$ , and  $\llbracket y_i \rrbracket_1^B \leftarrow y_i$ ; and compute  $\llbracket \text{lt}_i \rrbracket^B \leftarrow \llbracket \neg x_i \rrbracket^B \wedge \llbracket y_i \rrbracket^B$ 
   by half-AND (see Section 2.5)
7: end for
8: for  $i = n - 1, \dots, 2, 1$  do
9:   Locally compute  $\llbracket \text{neq}_{[n-1,i]} \rrbracket^A \leftarrow \sum_{j=i}^{n-1} \llbracket \text{neq}_j \rrbracket^A$ 
10:   $P_0$  parses  $\llbracket \text{neq}_{[n-1,i]} \rrbracket_0^A$  as  $(\nu_{i,0,n_0-1} \cdots \nu_{i,0,1} \nu_{i,0,0})_2$ , and  $P_1$  parses  $(n - \llbracket \text{neq}_{[n-1,i]} \rrbracket_1^A) \bmod n$  as
    $(\nu_{i,1,n_0-1} \cdots \nu_{i,1,1} \nu_{i,1,0})_2$ 
11: end for
12: for  $i = n - 2, \dots, 1, 0$  do
13:   Prepare (in the offline phase) an  $(n_0 + 1)$ -BTE  $(\llbracket a_{i,s} \rrbracket^B)_{\emptyset \neq s \subseteq [n_0+1]}$  (see Section 2.6)
14:   for  $j = n_0 - 1, \dots, 1, 0$  do
15:     Locally set  $\llbracket \text{eq}_{i+1,j} \rrbracket_0^B \leftarrow \nu_{i+1,0,j} \oplus 1$  and  $\llbracket \text{eq}_{i+1,j} \rrbracket_1^B \leftarrow \nu_{i+1,1,j}$ ; and compute  $\langle \text{eq}_{i+1,j} \rangle \leftarrow$ 
       $\text{bit2T}(\llbracket \text{eq}_{i+1,j} \rrbracket^B; \llbracket a_{i,\{j\}} \rrbracket^B)$ 
16:   end for
17:   Compute  $\langle \text{lt}_i \rangle \leftarrow \text{bit2T}(\llbracket \text{lt}_i \rrbracket^B; \llbracket a_{i,\{n_0\}} \rrbracket^B)$ 
18: end for
19: By using  $(n_0 + 1)$ -fan-in AND with the BTE, locally compute

```

$$\llbracket \text{lt} \rrbracket^B \leftarrow \llbracket \text{lt}_{n-1} \rrbracket^B \oplus \bigoplus_{i=0}^{n-2} \left(\bigwedge_{j=0}^{n_0-1} \langle \text{eq}_{i+1,j} \rangle \wedge \langle \text{lt}_i \rangle \right)$$

```

20: output  $\llbracket \text{lt} \rrbracket^B$ 

```

4.3 Analysis of Bit Complexity for Protocol 1

Here we describe the bit complexity of Protocol 1.

4.3.1 Online Bit Complexity

In Protocol 1, online communication occurs only in Lines 3, 6, 15, and 17. As for the number of online communication bits, we have the following:

- For Line 3, each bit2A requires 2-bit communication, therefore the total number of communication bits is $2(n - 1)$ bits.
- For Line 6, each half-AND requires 2-bit communication, therefore the total number of communication bits is $2n$ bits.
- For Line 15, each bit2T requires 2-bit communication, therefore the total number of communication bits is $2(n - 1)n_0$ bits.
- For Line 17, each bit2T requires 2-bit communication, therefore the total number of communication bits is $2(n - 1)$ bits.

Summarizing, the total number of online communication bits is

$$\begin{aligned}
2(n-1) + 2n + 2(n-1)n_0 + 2(n-1) &= 2(n-1)n_0 + 6n - 4 \\
&= 2(n-1)\lceil \log_2 n \rceil + 6n - 4 \\
&\sim 2n \log_2 n \text{ (bits)}.
\end{aligned}$$

4.3.2 Offline Bit Complexity

As for the offline communication cost, we have the following:

- For Line 3, each bit2A requires $\binom{2}{1}\text{-OT}_{n_0}^1$ in the offline phase. Therefore the total offline cost is $\binom{2}{1}\text{-OT}_{n_0}^{n-1}$.
- For Line 6, each half-AND requires $\binom{2}{1}\text{-OT}_1^1$ in the offline phase. Therefore the total offline cost is $\binom{2}{1}\text{-OT}_1^n$.
- For each $i = n-2, \dots, 1, 0$, to generate an (n_0+1) -BTE $([a_{i,S}]^B)_{\emptyset \neq S \subseteq [n_0+1]}$, it requires $\binom{2^{n_0+1}}{1}\text{-OT}_{2^{n_0+1}-n_0-2}^1$ in the offline phase. Therefore the total offline cost is $\binom{2^{n_0+1}}{1}\text{-OT}_{2^{n_0+1}-n_0-2}^{n-1}$.

Summarizing, the total offline communication cost is $\binom{2}{1}\text{-OT}_{n_0}^{n-1} + \binom{2}{1}\text{-OT}_1^n + \binom{2^{n_0+1}}{1}\text{-OT}_{2^{n_0+1}-n_0-2}^{n-1}$, which is (according to Section 2.4) equivalent to communicating the following number of bits:

$$\begin{aligned}
&(n-1)(2n_0+1) + n \cdot 3 + (n-1)(2^{n_0+1}(2^{n_0+1}-n_0-2) + n_0+1) \\
&= (n-1)(3n_0+2) + 3n + (n-1)2^{2n_0+2} - (n-1)2^{n_0+1}(n_0+2) \\
&\approx (n-1)(3\log_2 n + 2) + 3n + 4(n-1)n^2 - 2(n-1)n(\log_2 n + 2) \\
&= 4n^3 - 4n^2 + 3n - (n-1)(2n\log_2 n + 4n - 3\log_2 n - 2) \\
&< 4n^3 \text{ (bits)}
\end{aligned}$$

and setting up a VOLE correlation of dimension

$$d := (n-1) \cdot 1 + n \cdot 1 + (n-1)(n_0+1) = n(n_0+3) - n_0 - 2 \lesssim n(\log_2 n + 3).$$

Hence the total number of offline communication bits is

$$\approx 4n^3 + 2\mu + \lambda + (\mu + \lambda) \log_2 d \approx 4n^3 + 2\mu + \lambda + (\mu + \lambda)(\log_2 n + \log_2^2 n) \text{ (bits)}.$$

5 Our Second Proposed Comparison Protocol $\Pi_{\text{cmp},2}$

Our second proposed protocol is also based on the construction in Section 4.1, but we use the internal equality testing protocol in Section 3.2. In this instantiation, for each $i = n-2, \dots, 1, 0$, the $\llbracket^B$ -share $[Z_{i+1}]^B$ is computed as follows:

1. For each $j = 0, 1, \dots, n-1$:
 - P_0 sets $[\text{eq}_{i+1,0,j}]_0^B \leftarrow \mathbf{1}\{[\text{neq}_{[n-1,i+1]}]_0^A = j\}$, and P_1 sets $[\text{eq}_{i+1,0,j}]_1^B \leftarrow 0$.
 - P_0 sets $[\text{eq}_{i+1,1,j}]_0^B \leftarrow 0$, and P_1 sets $[\text{eq}_{i+1,1,j}]_1^B \leftarrow \mathbf{1}\{n - [\text{neq}_{[n-1,i+1]}]_1^A \bmod n = j\}$.
2. The parties compute $[Z_{i+1}]^B \leftarrow \bigoplus_{j=0}^{n-1} ([\text{eq}_{i+1,0,j}]^B \wedge [\text{eq}_{i+1,1,j}]^B)$.

Protocol 2 Our Second Two-Round Plaintext Comparison Protocol $\Pi_{\text{cmp},2}$

Input: P_0 has input $x = (x_{n-1} \cdots x_1 x_0)_2$, and P_1 has input $y = (y_{n-1} \cdots y_1 y_0)_2$

Output: $\llbracket^B\text{-share } [1\{x < y\}]\rrbracket^B$ of the bit $1\{x < y\}$

- 1: **for** $i = n - 1, \dots, 2, 1$ **do**
- 2: Locally set $[\text{neq}_i]_0^B \leftarrow x_i$ and $[\text{neq}_i]_1^B \leftarrow y_i$
- 3: Compute $[\text{neq}_i]^A \leftarrow \text{bit2A}([\text{neq}_i]^B)$ where the plaintext space for $[\text{neq}_i]^A$ is set to $\mathbb{Z}_n$
- 4: **end for**
- 5: **for** $i = n - 1, \dots, 2, 1$ **do**
- 6: Locally compute $[\text{neq}_{[n-1,i]}]^A \leftarrow \sum_{j=i}^{n-1} [\text{neq}_j]^A$
- 7: **end for**
- 8: **for** $i = n - 2, \dots, 1, 0$ **do**
- 9: **for** $j = 0, 1, \dots, n - 1$ **do**
- 10: P_0 sets

$$[\varepsilon_{i,0,j}]_0^B \leftarrow 1\{[\text{neq}_{[n-1,i+1]}]_0^A = j\} \wedge (x_i \oplus 1) ,$$
 and P_1 sets $[\varepsilon_{i,0,j}]_1^B \leftarrow 0$
- 11: P_0 sets $[\varepsilon_{i,1,j}]_0^B \leftarrow 0$, and P_1 sets

$$[\varepsilon_{i,1,j}]_1^B \leftarrow 1\{(n - [\text{neq}_{[n-1,i+1]}]_1^A) \bmod n = j\} \wedge y_i$$
- 12: **end for**
- 13: **end for**
- 14: By using half-AND (see Section 2.5), compute

$$[\text{lt}]^B \leftarrow [\text{lt}_{n-1}]^B \oplus \bigoplus_{i=0}^{n-2} \bigoplus_{j=0}^{n-1} ([\varepsilon_{i,0,j}]^B \wedge [\varepsilon_{i,1,j}]^B)$$

- 15: **output** $[\text{lt}]^B$
-

Now we observe (by using the terminology in Definition 1) that $[\text{eq}_{i+1,0,j}]^B$ is a P_0 -side trivial $\llbracket^B\text{-share}$, $[\text{eq}_{i+1,1,j}]^B$ is a P_1 -side trivial $\llbracket^B\text{-share}$, and $[\text{lt}_i]^B$ is the AND of a P_0 -side trivial $\llbracket^B\text{-share}$ $[\neg x_i]^B$ and a P_1 -side trivial $\llbracket^B\text{-share}$ $[y_i]^B$. Moreover, for two P_0 -side trivial $\llbracket^B\text{-shares}$ $[X]^B = (X, 0)$ and $[Y]^B = (Y, 0)$, the $\llbracket^B\text{-share}$ $(X \wedge Y, 0)$ can be used as a result of $[X]^B \wedge [Y]^B$, which can be obtained locally. The same holds for the AND of two P_1 -side trivial $\llbracket^B\text{-shares}$. Hence, to compute $[Z_{i+1}]^B \wedge [\text{lt}_i]^B$, we can first compute $[\text{eq}_{i+1,0,j}]^B \wedge [\neg x_i]^B$ and $[\text{eq}_{i+1,1,j}]^B \wedge [y_i]^B$ locally and then compute their AND by using a (not multi-fan-in) half-AND:

$$\begin{aligned} [\text{eq}_{i+1,0,j}]^B \wedge [\text{eq}_{i+1,1,j}]^B \wedge [\text{lt}_i]^B &= [\text{eq}_{i+1,0,j}]^B \wedge [\text{eq}_{i+1,1,j}]^B \wedge ([\neg x_i]^B \wedge [y_i]^B) \\ &= ([\text{eq}_{i+1,0,j}]^B \wedge [\neg x_i]^B) \wedge ([\text{eq}_{i+1,1,j}]^B \wedge [y_i]^B) \end{aligned}$$

and the two terms in the parentheses in the right-hand side can be locally computed and are again trivial $\llbracket^B\text{-shares}$. Based on the argument, we obtain our second two-round protocol $\Pi_{\text{cmp},2}$ as in Protocol 2.

5.1 Analysis of Bit Complexity for Protocol 2

Here we describe the bit complexity of Protocol 2.

5.1.1 Online Bit Complexity

In Protocol 2, online communication occurs only in Lines 3 and 14. As for the number of online communication bits, we have the following:

- For Line 3, each bit2A requires 2-bit communication, therefore the total number of communication bits is $2(n-1)$ bits.
- For Line 14, each half-AND requires 2-bit communication, therefore the total number of communication bits is $2(n-1)n$ bits.

Summarizing, the total number of online communication bits is

$$2(n-1) + 2(n-1)n = 2(n-1)(n+1) = 2n^2 - 2 \sim 2n^2 \text{ (bits)}.$$

5.1.2 Offline Bit Complexity

As for the offline communication cost, we have the following:

- For Line 3, each bit2A requires $\binom{2}{1}\text{-OT}_{n_0}^1$ (where $n_0 := \lceil \log_2 n \rceil$) in the offline phase. Therefore the total offline cost is $\binom{2}{1}\text{-OT}_{n_0}^{n-1}$.
- For Line 14, each half-AND requires $\binom{2}{1}\text{-OT}_1^1$ in the offline phase. Therefore the total offline cost is $\binom{2}{1}\text{-OT}_1^{(n-1)n}$.

Summarizing, the total offline communication cost is $\binom{2}{1}\text{-OT}_{n_0}^{n-1} + \binom{2}{1}\text{-OT}_1^{(n-1)n}$, which is (according to Section 2.4) equivalent to communicating the following number of bits:

$$\begin{aligned} (n-1)(2n_0+1) + (n-1)n \cdot 3 &= 2nn_0 + 3n^2 - 2n - 2n_0 - 1 \\ &\lesssim 3n^2 + 2n \log_2 n \text{ (bits)} \end{aligned}$$

and setting up a VOLE correlation of dimension

$$d := (n-1) \cdot 1 + (n-1)n \cdot 1 = n^2 - 1 < n^2.$$

Hence the total number of offline communication bits is

$$\begin{aligned} &\approx 3n^2 + 2n \log_2 n + 2\mu + \lambda + (\mu + \lambda) \log_2 d \\ &< 3n^2 + 2n \log_2 n + 2\mu + \lambda + (\mu + \lambda) \cdot 2 \log_2 n \\ &= 3n^2 + 2(n + \mu + \lambda) \log_2 n + 2\mu + \lambda \text{ (bits)}. \end{aligned}$$

6 Application Case Study: Secure Deep Learning Inference

In this section, we show an application case study for our proposed protocols focusing on secure deep learning inference.

6.1 Communication Cost Analysis

To demonstrate the end-to-end practicality and trade-offs of our two-round comparison protocols, we estimate the total communication costs (both offline and online) and online round complexity for a privacy-preserving neural network inference benchmark evaluated in ABY2.0 [19]. We consider a fully-connected neural network (FCNN) with two hidden layers, each containing 128 nodes, and an output layer with 10 nodes, which requires 256 ReLU activations in total.

In this 2PC inference setting, the linear (fully-connected) layers are evaluated using the efficient scalar product protocol of ABY2.0. Based on the parameter settings of $\ell = 64$ and $\kappa = 128$, each underlying multiplication requires 48 bytes of offline communication during the setup phase and exactly 2 ring elements (16 bytes) in the online phase. Given the network architecture, the linear layers require a total of 118,016 multiplication triples, resulting in 5,532.0 KiB (approx. 5.40 MiB) of offline communication and 4.16 KiB of online communication across 3 online rounds (1 round per layer).

Table 5: Estimated total communication cost and rounds for 3-layer NN inference (FCNN with 256 ReLUs).

Protocol	Offline	Online	
	Communication	Communication	# of rounds
Lu et al. [11]	9.01 MiB	40.9 KiB	11
Ours $\Pi_{\text{cmp},1}$ (§ 4)	38.28 MiB	43.7 KiB	9
Ours $\Pi_{\text{cmp},2}$ (§ 5)	6.11 MiB	264.2 KiB	9

For the non-linear activation layers, evaluating each ReLU function $\text{ReLU}(v) = \max(0, v)$ requires first invoking the secure comparison (LT) protocol to extract the sign bit $u \in \{0, 1\}$, followed by the PV protocol to securely compute the multiplication uv [19]. According to the concrete cost specified in ABY2.0, the PV protocol requires $5(\kappa + \ell)$ bits (120 bytes) of offline setup communication and 2 ring elements (16 bytes) of online communication per invocation. Consequently, the 256 PV conversions consume an additional 30.0 KiB of offline communication and 4.00 KiB of online communication across both layers.

Table 5 summarizes the comprehensive communication costs by integrating the linear layers, the LT protocols, and the subsequent PV conversions. This holistic analysis highlights a clear and meaningful trade-off between our two proposed protocols. Our first protocol, $\Pi_{\text{cmp},1}$, targets applications where online latency is paramount; it successfully reduces the total network inference from 11 rounds to 9 rounds (an 18.2% reduction in round complexity) with a minimal online communication volume of 43.66 KiB, which is highly competitive with the baseline using Lu et al.’s protocol [11] (40.91 KiB). While $\Pi_{\text{cmp},1}$ incurs an inflated offline cost (38.28 MiB) due to the use of larger fan-in AND gates, it directly mitigates the network latency bottleneck in multi-layer deep learning inference.

Conversely, our second protocol, $\Pi_{\text{cmp},2}$, is designed to optimize the overall bandwidth consumption. While it increases the online communication to 264.16 KiB, it suppresses the offline cost of the comparison layers. Remarkably, when considering the total communication volume (offline + online), $\Pi_{\text{cmp},2}$ requires only 6.37 MiB in total, achieving approximately a 30% reduction in total network bandwidth compared to the baseline using Lu et al.’s protocol (9.05 MiB). This case study underscores that our protocols offer flexible options depending on whether the deployment environment is constrained by network latency or total bandwidth availability.

6.2 Trade-off Analysis and Practical Boundaries

To evaluate the practical impact of network characteristics, we simulate the end-to-end execution latency across two network environments with four distinct settings (evaluating both online-only and total latencies for both LAN and WAN) based on the network parameters evaluated in Section 6 of [19] (LAN: 25 Gbps, 0.056 ms RTT; WAN: 75 Mbps, 60.19 ms RTT). Since local CPU evaluation times for secret-sharing operations are known to be negligible (on the order of microseconds), the network transfer and propagation time strictly dominate the actual runtime. As illustrated in Figure 1, the optimal protocol choice depends tightly on the deployment environment and phase constraints:

- **Online-Only Latency:** In both LAN and WAN environments, our protocols outperform Lu et al.’s protocol by shifting the round complexity from 11 to 9 rounds. Specifically, in the WAN environment, $\Pi_{\text{cmp},1}$ minimizes the online latency to 546.4 ms, successfully mitigating the network RTT bottleneck.
- **Total Latency (Offline + Online) Boundaries:** When taking the offline phase into account, a clear technical boundary emerges. Due to the $O(n^3)$ offline communication overhead, our first protocol $\Pi_{\text{cmp},1}$ is less suitable for total time evaluation, requiring 4728 ms under WAN constraints, where it is outperformed by Lu et al.’s protocol (1651 ms).
- **The Superiority of $\Pi_{\text{cmp},2}$:** To eliminate this weak point of $\Pi_{\text{cmp},1}$, our second protocol $\Pi_{\text{cmp},2}$ demonstrates its practicality. By compressing the offline data volume, $\Pi_{\text{cmp},2}$ achieves the fastest total exe-

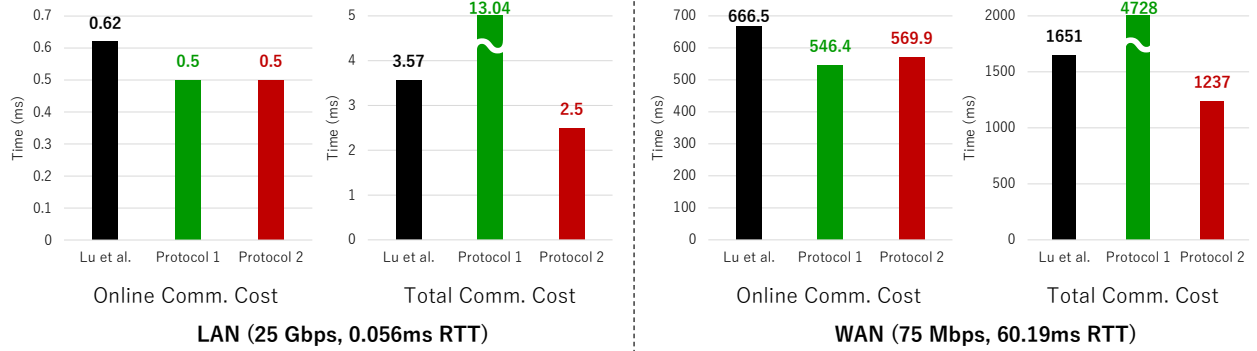


Figure 1: Trade-off of communication costs between existing and our protocols.

cution time in both LAN (2.50 ms) and WAN (1237 ms) environments, providing a 25% total speedup over Lu et al.’s protocol under WAN environments.

7 Conclusion

In this work, we presented two novel secure comparison protocols over plaintexts, achieving a constant two rounds. While our contributions significantly advance the state-of-the-art, a challenging direction remains for future research; that is, further reducing the round complexity. This task appears non-trivial. Regarding the first challenge, our two-round constructions (Protocols 1 and 2) fundamentally rely on a two-round equality testing framework. Achieving a one-round comparison protocol would likely necessitate a one-round equality testing primitive—a feat that surpasses the inherent two-round barrier of GC—or a complete departure from the “leftmost differing bit” strategy employed in this work.

References

- [1] Donald Beaver. Efficient multiparty protocols using circuit randomization. In Joan Feigenbaum, editor, *Advances in Cryptology - CRYPTO ’91, 11th Annual International Cryptology Conference, Santa Barbara, California, USA, August 11-15, 1991, Proceedings*, volume 576 of *Lecture Notes in Computer Science*, pages 420–432. Springer, 1991.
- [2] Elette Boyle, Nishanth Chandran, Niv Gilboa, Divya Gupta, Yuval Ishai, Nishant Kumar, and Mayank Rathee. Function secret sharing for mixed-mode and fixed-point secure computation. In Anne Canteaut and François-Xavier Standaert, editors, *Advances in Cryptology – EUROCRYPT 2021, Part II*, volume 12697 of *Lecture Notes in Computer Science*, pages 871–900, Zagreb, Croatia, October 17–21, 2021. Springer, Cham, Switzerland.
- [3] Elette Boyle, Geoffroy Couteau, Niv Gilboa, Yuval Ishai, Lisa Kohl, Peter Rindal, and Peter Scholl. Efficient two-round OT extension and silent non-interactive secure computation. In Lorenzo Cavallaro, Johannes Kinder, XiaoFeng Wang, and Jonathan Katz, editors, *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security, CCS 2019, London, UK, November 11-15, 2019*, pages 291–308. ACM, 2019.
- [4] Jung Hee Cheon, Dongwoo Kim, and Duhyeong Kim. Efficient homomorphic comparison methods with optimal complexity. In Shiho Moriai and Huaxiong Wang, editors, *Advances in Cryptology - ASIACRYPT 2020 - 26th International Conference on the Theory and Application of Cryptology and Information Security, Daejeon, South Korea, December 7-11, 2020, Proceedings, Part II*, volume 12492 of *Lecture Notes in Computer Science*, pages 221–256. Springer, 2020.

- [5] Geoffroy Couteau. New protocols for secure equality test and comparison. In Bart Preneel and Frederik Vercauteren, editors, *ACNS 2018: 16th International Conference on Applied Cryptography and Network Security*, volume 10892 of *Lecture Notes in Computer Science*, pages 303–320, Leuven, Belgium, July 2–4, 2018. Springer, Cham, Switzerland.
- [6] Daniel Demmler, Thomas Schneider, and Michael Zohner. ABY - A framework for efficient mixed-protocol secure two-party computation. In *ISOC Network and Distributed System Security Symposium – NDSS 2015*, San Diego, CA, USA, February 8–11, 2015. The Internet Society.
- [7] Ye Dong, Xudong Chen, Xiangfu Song, Yaxi Yang, Wen-Jie Lu, Tianwei Zhang, Jianying Zhou, and Jin-Song Dong. Alkaid: Accelerating three-party boolean circuits by mixing correlations and redundancy. *IEEE Transactions on Information Forensics and Security*, 21:640–654, 2026.
- [8] Juan A. Garay, Berry Schoenmakers, and José Villegas. Practical and secure solutions for integer comparison. In Tatsuaki Okamoto and Xiaoyun Wang, editors, *PKC 2007: 10th International Conference on Theory and Practice of Public Key Cryptography*, volume 4450 of *Lecture Notes in Computer Science*, pages 330–342, Beijing, China, April 16–20, 2007. Springer Berlin Heidelberg, Germany.
- [9] Oded Goldreich. *The Foundations of Cryptography - Volume 2: Basic Applications*. Cambridge University Press, 2004.
- [10] Xiaojie Guo, Kang Yang, Xiao Wang, Wenhao Zhang, Xiang Xie, Jiang Zhang, and Zheli Liu. Half-tree: Halving the cost of tree expansion in COT and DPF. In Carmit Hazay and Martijn Stam, editors, *Advances in Cryptology – EUROCRYPT 2023, Part I*, volume 14004 of *Lecture Notes in Computer Science*, pages 330–362, Lyon, France, April 23–27, 2023. Springer, Cham, Switzerland.
- [11] Tianpei Lu, Xin Kang, Bingsheng Zhang, Zhuo Ma, Xiaoyuan Zhang, Yang Liu, Kui Ren, and Chun Chen. Efficient 2pc for constant round secure equality testing and comparison. In Lujo Bauer and Giancarlo Pellegrino, editors, *34th USENIX Security Symposium, USENIX Security 2025, Seattle, WA, USA, August 13-15, 2025*, pages 1985–2004. USENIX Association, 2025.
- [12] Payman Mohassel and Yupeng Zhang. SecureML: A system for scalable privacy-preserving machine learning. In *2017 IEEE Symposium on Security and Privacy, SP 2017, San Jose, CA, USA, May 22-26, 2017*, pages 19–38. IEEE Computer Society, 2017.
- [13] Hiraku Morita, Nuttapong Attrapadung, Tadanori Teruya, Satsuya Ohata, Koji Nuida, and Goichiro Hanaoka. Constant-round client-aided secure comparison protocol. In Javier López, Jianying Zhou, and Miguel Soriano, editors, *Computer Security - 23rd European Symposium on Research in Computer Security, ESORICS 2018, Barcelona, Spain, September 3-7, 2018, Proceedings, Part II*, volume 11099 of *Lecture Notes in Computer Science*, pages 395–415. Springer, 2018.
- [14] Moni Naor and Benny Pinkas. Oblivious transfer and polynomial evaluation. In Jeffrey Scott Vitter, Lawrence L. Larmore, and Frank Thomson Leighton, editors, *Proceedings of the Thirty-First Annual ACM Symposium on Theory of Computing, May 1-4, 1999, Atlanta, Georgia, USA*, pages 245–254. ACM, 1999.
- [15] Moni Naor and Benny Pinkas. Efficient oblivious transfer protocols. In S. Rao Kosaraju, editor, *Proceedings of the Twelfth Annual Symposium on Discrete Algorithms, January 7-9, 2001, Washington, DC, USA*, pages 448–457. ACM/SIAM, 2001.
- [16] Takashi Nishide and Kazuo Ohta. Multiparty computation for interval, equality, and comparison without bit-decomposition protocol. In Tatsuaki Okamoto and Xiaoyun Wang, editors, *Public Key Cryptography - PKC 2007, 10th International Conference on Practice and Theory in Public-Key Cryptography, Beijing, China, April 16-20, 2007, Proceedings*, volume 4450 of *Lecture Notes in Computer Science*, pages 343–360. Springer, 2007.

- [17] Satsuya Ohata and Koji Nuida. Communication-efficient (client-aided) secure two-party protocols and its application. In Joseph Bonneau and Nadia Heninger, editors, *Financial Cryptography and Data Security - 24th International Conference, FC 2020, Kota Kinabalu, Malaysia, February 10-14, 2020 Revised Selected Papers*, volume 12059 of *Lecture Notes in Computer Science*, pages 369–385. Springer, 2020.
- [18] Arpita Patra, Thomas Schneider, Ajith Suresh, and Hossein Yalame. ABY2.0: Improved mixed-protocol secure two-party computation. Cryptology ePrint Archive, Report 2020/1225, 2020.
- [19] Arpita Patra, Thomas Schneider, Ajith Suresh, and Hossein Yalame. ABY2.0: improved mixed-protocol secure two-party computation. In Michael D. Bailey and Rachel Greenstadt, editors, *30th USENIX Security Symposium, USENIX Security 2021, August 11-13, 2021*, pages 2165–2182. USENIX Association, 2021.
- [20] Deevashwer Rathee, Mayank Rathee, Nishant Kumar, Nishanth Chandran, Divya Gupta, Aseem Rastogi, and Rahul Sharma. CryptFlow2: Practical 2-party secure inference. In Jay Ligatti, Xinming Ou, Jonathan Katz, and Giovanni Vigna, editors, *CCS '20: 2020 ACM SIGSAC Conference on Computer and Communications Security, Virtual Event, USA, November 9-13, 2020*, pages 325–342. ACM, 2020.
- [21] Sijun Tan, Brian Knott, Yuan Tian, and David J. Wu. Cryptgpu: Fast privacy-preserving machine learning on the GPU. In *42nd IEEE Symposium on Security and Privacy, SP 2021, San Francisco, CA, USA, 24-27 May 2021*, pages 1021–1038. IEEE, 2021.
- [22] Xiao Wang, Alex J. Malozemoff, and Jonathan Katz. EMP-toolkit: Efficient MultiParty computation toolkit. <https://github.com/emp-toolkit>, 2016.
- [23] Andrew Chi-Chih Yao. Protocols for secure computations (extended abstract). In *23rd Annual Symposium on Foundations of Computer Science, Chicago, Illinois, USA, 3-5 November 1982*, pages 160–164. IEEE Computer Society, 1982.
- [24] Andrew Chi-Chih Yao. How to generate and exchange secrets (extended abstract). In *27th Annual Symposium on Foundations of Computer Science, Toronto, Canada, 27-29 October 1986*, pages 162–167. IEEE Computer Society, 1986.

A OT from VOLE Correlations

By a VOLE correlation with parameters p (prime), m , and d (positive integers), we mean random vectors $\vec{\rho} = (\rho_0, \dots, \rho_{d-1}) \in (\mathbb{F}_p)^d$ and $\vec{w} = (w_0, \dots, w_{d-1})$, $\vec{w}' = (w'_0, \dots, w'_{d-1}) \in (\mathbb{F}_{p^m})^d$ and a random scalar $\sigma \in \mathbb{F}_{p^m}$ with the relation $\vec{w}' = \sigma \cdot \vec{\rho} + \vec{w}$, where one party holds $(\sigma, \vec{w}')$ and the other party holds $(\vec{\rho}, \vec{w})$. Here we typically consider the case of $p = 2$ and $m = \lambda$, while the dimension d should depend on the number of required OT executions.

Given such a VOLE correlation, a $\binom{k}{1}$ -OT $_\ell$ can be executed as follows, where the sender holds $(\sigma, \vec{w}')$, the receiver holds $(\vec{\rho}, \vec{w})$, and ct denotes a counter held by both parties (a variant of the method in [14]):

1. Let $x_0, x_1, \dots, x_{k-1} \in \{0, 1\}^\ell$ be the inputs of the sender, and let $r \in [k]$ be the input of the receiver. Set $\nu \leftarrow \lceil \log_2 k \rceil$. If $k < 2^\nu$, then the sender sets $x_i \leftarrow \vec{0}$ for each $i = k, k+1, \dots, 2^\nu - 1$.
2. For $i = 0, 1, \dots, \nu - 1$:
 - (a) The sender sets $v_{i,0} \leftarrow w'_{\text{ct}}$ and $v_{i,1} \leftarrow v_{i,0} - \sigma$. The receiver sets $\rho'_i \leftarrow \rho_{\text{ct}}$ and $v'_i \leftarrow w_{\text{ct}}$. (Note that now $v'_i = v_{i,\rho'_i}$.)
 - (b) Set $\text{ct} \leftarrow \text{ct} + 1$.
3. The receiver sets $\rho' \leftarrow (\rho'_{\nu-1} \cdots \rho'_1 \rho'_0)_2$ and $r' \leftarrow \rho' - r \bmod 2^\nu$, and sends r' to the sender.

4. For $i = 0, 1, \dots, 2^\nu - 1$:

- (a) The sender parses $r' + i \bmod 2^\nu$ as $(j_{\nu-1} \cdots j_1 j_0)_2$.
- (b) The sender computes $K_i \leftarrow H(v_{\nu-1, j_{\nu-1}} \parallel \cdots \parallel v_{1, j_1} \parallel v_{0, j_0} \parallel i) \in \{0, 1\}^\ell$ where H is a cryptographic hash function (modeled as a random oracle), sets $x'_i \leftarrow K_i \oplus x_i$, and sends x'_i to the receiver.

5. The receiver outputs $H(v'_{\nu-1} \parallel \cdots \parallel v'_1 \parallel v'_0 \parallel r) \oplus x'_r$.

The number of communication bits of this protocol (given a VOLE correlation) is $2^{\lceil \log_2 k \rceil} \ell + \lceil \log_2 k \rceil$ bits, while it spends $\lceil \log_2 k \rceil$ dimensions for a VOLE correlation. On the other hand, to generate a VOLE correlation of dimension d according to [3], the communication cost is $\lceil \log_2 cd \rceil = \lceil \log_2 d + \log_2 c \rceil \approx \log_2 d$ executions of $\binom{2}{1}\text{-OT}_\lambda$ for a small constant $c > 1$ and λ -bit communication. When the latter $\binom{2}{1}\text{-OT}_\lambda$'s are executed with the method of [15], the number of communication bits for the $\log_2 d$ executions is $2\mu + (\mu + \lambda) \log_2 d$ where $\mu = \mu_\lambda$ denotes the bit length of an element of a group for which the CDH assumption holds. Hence the number of communication bits for generating a VOLE correlation is approximately $2\mu + \lambda + (\mu + \lambda) \log_2 d$ bits.